

LEAN AUTOMATION / CONCRETE ALGORITHMS

Forge

Finite Certificates for Infinite Behaviors

Inductive ideals, weighted residuals,
and pole-safe creative telescoping

Technical research report for Vladimir Reshetnikov
15 September 2026

Three executable search workers extend the reviewed Forge design with non-conserved polynomial invariants, equivalence of rational weighted representations, and certified recurrences for binomial sums. The package contains exact certificates, an independently implemented standard-library replayer, and 318 passing tests. The Lean proof principles and replay targets are supplied as *uncompiled specimens*; no implemented Lean tactic or head-to-head tactic improvement is claimed.

Repository baseline: Forge 674521027d96.
Original code and report source are included. No third-party repository, font, or solver distribution is bundled.

Abstract

Forge already proposes a proof-planning layer above Lean’s `grind`, including induction generalization, arithmetic witnesses, nonlinear certificates, and careful evidence management. This report develops three additional algorithmic capabilities rather than restating that architecture. First, a descending sequence of finite-dimensional polynomial spaces discovers jointly inductive ideals with polynomial transition multipliers. Second, reachable-space closure proves equivalence of rational weighted representations, or returns a concrete distinguishing word. Third, a bounded creative-telescoping search synthesizes recurrences together with regularized, pole-safe summation certificates. Each worker has an explicit source fragment, a finite acceptance language, a soundness argument, a sharply delimited completeness statement, and an implementation path into Lean. The executed corpus contains 27 invariant certificates, 23 weighted closure certificates, 25 distinguishing words, and three telescopers; four bounded searches return unknown. Independent replay works with Python’s site packages disabled. The report separates these results from the still-unimplemented Lean front ends, formalized checker theorems, and end-to-end tactic benchmarks.

Contents

1	The increment relative to the inspected repository	3
1.1	What is already present, and therefore not a contribution here	3
1.2	Pinned inputs and evidence levels	3
1.3	A common target: finite closure of an infinite behavior	4
1.4	Representative goals	4
2	Worker I: discover jointly inductive polynomial ideals	5
2.1	From conservation to preservation of a zero set	5
2.2	Input fragment and semantics	5
2.3	The descending-space algorithm	6
2.4	What kind of completeness is obtained?	7
2.5	Multiplier extraction without trusting the Gröbner engine	8
2.6	The finite certificate and its soundness	8
2.7	Examples that discriminate between mechanisms	9
2.8	Implementation refinements worth making next	9
3	Worker II: finite residual closure for weighted behaviors	10
3.1	Why a recursive equality can be linear algebra	10
3.2	Reachable-space search	10
3.3	The closure certificate	11
3.4	Building representations from code	12
3.5	Worked example: a product of counts versus subsequence counts	13
3.6	Counterexamples, minimization, and use during synthesis	13
3.7	Cost model and boundaries	13
4	Worker III: pole-safe creative telescoping	14
4.1	Beyond polynomial antidifferences	14
4.2	A fully explicit bounded ansatz	14
4.3	Where the apparent poles come from	15
4.4	Deriving the interior identity without illegal cancellation	16
4.5	Boundary equations are part of the certificate	16
4.6	A recurrence is not yet an equality with a proposed sequence	17
4.7	Three synthesized certificates	17
4.8	A route to a broader summation worker	19

5	Concrete integration into Forge, Leant, and Djex	19
5.1	Reuse the existing controller; add three semantic front ends	19
5.2	Polynomial extraction and replay	20
5.3	Weighted extraction and replay	20
5.4	Summation extraction and replay	21
5.5	A practical worker transaction	21
5.6	Concrete library choices	22
5.7	Leant and Djex: add behavior domains, not another candidate authority	22
5.8	A targeted composition experiment for the remaining indexing gap	23
6	Executed experiments and what they establish	24
6.1	Recorded environment and reproducibility	24
6.2	Invariant corpus and ablation	24
6.3	Weighted corpus	25
6.4	Telescoping corpus	25
6.5	Mutation and differential tests	25
6.6	Recorded costs, without a tactic-speed claim	26
6.7	The remaining evidence gap	26
7	An implementation order with falsifiable acceptance gates	27
7.1	First gate: one complete weighted proof in Lean	27
7.2	Second gate: a non-conserved invariant from source code	27
7.3	Third gate: the square sum with all endpoints	27
7.4	Controller integration and proof export	28
7.5	Evaluation after the formalized slice exists	28
7.6	Further directions that fit the same boundaries	28
7.7	Assessment	29
A	Artifact map and reproduction	29
A.1	Files and their roles	29
A.2	Commands	30
B	Certificate data contracts	30
B.1	Rationals and sparse polynomials	30
B.2	Inductive-ideal data	31
B.3	Weighted data	31
B.4	Telescoping data	31
C	Audit scope and source provenance	31
	References	32

1 The increment relative to the inspected repository

1.1 What is already present, and therefore not a contribution here

The reviewed Forge snapshot is a consolidated design and prototype collection, not an installed Lean tactic. Its article already covers conclusion-directed retrieval, structural induction and accumulator generalization, auxiliary-lemma search, polynomial recurrence invariants, affine and integer witnesses, nonlinear certificates, and Boolean/theory cooperation. It also incorporates a substantial review of Leant and Djex, including their verification boundaries and behavioral-evidence distinctions [1, 2, 3, 4].

Consequently, “add induction”, “use a computer algebra system”, “produce certificates”, and “reuse Djex” would not answer the present engineering question. This report instead supplies three concrete workers and explains the mathematics and interfaces required to turn their finite outputs into proofs of unbounded behavior.

Area	Reviewed Forge design	Increment developed here
Polynomial invariants	Polynomial additive recurrences and conserved quantities; general ideal multipliers mentioned as a next step	An executable descending ideal-preimage algorithm, tracked multiplier extraction, and a greatest-fixed-point theorem for a specified bounded generator class.
Recursive equivalence	Induction, rewriting, and lemma discovery	A finite reachable-space decision procedure for rational weighted representations, including tensor-composed observations and checkable counterexample words.
Finite sums	Polynomial finite-difference invariants	A concrete creative-telescoping ansatz for hypergeometric binomial-power sums, including moving support, pole regularization, boundary checks, and recurrence uniqueness.
Leant/Djex integration	Typed candidates and checked behavioral domains	Proposed polynomial-state and weighted-observation domains tied to the existing source-owned candidate graphs; a separate composition experiment for blocked higher-order synthesis.

This is an extension relative to the *reviewed consolidated design*, not a claim that no analogous phrase occurs in any historical proposal or that the underlying mathematical subjects are new. Polynomial-ideal invariant generation, weighted-automaton equivalence, and creative telescoping all have substantial prior literature [9, 10, 12]. The contribution here is the explicit selection of useful fragments, executable algorithms, finite certificate formats, source-to-Lean obligations, and tests tailored to Forge.

1.2 Pinned inputs and evidence levels

Repository identities are recorded in `results/status.json`. The relevant snapshots are:

Repository	Commit
Forge	674521027d968d59f7b83220ed52304a85cb55e2
Leant	6bf05ad78c467989e68290f2d08bbbed40802d485
Djex	e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5

The Forge README records Lean 4.34.0 and Mathlib 1cf325a0cf67aca2b04d76b5380ff6a9e410aefa as its baseline. It also records three elaborated Lean-core files, which must not be confused with compiled certificate workers [1]. The experiments in this package used Python 3.13.5 and SymPy 1.14.0. Neither Lean nor Lake was available in the execution runtime. The supplied Lean specimens therefore have status NOT_RUN.

Four kinds of statement occur in the rest of the report. A theorem accompanied by a written proof is a mathematical claim about an explicitly defined algorithm or certificate language. A recorded experiment is an observation about the included Python programs. An integration design identifies code still to write. A compiled Lean proof would be a fourth, stronger artifact; this package does not contain one. This distinction is especially important when a Python checker verifies a polynomial identity: the identity is useful proof data, but the checker invocation is not itself a Lean theorem.

1.3 A common target: finite closure of an infinite behavior

The three workers share a useful mathematical pattern. Instead of expanding an unbounded sequence of recursive calls, find a finite object that is closed under the operation generating those calls.

For invariants, the object is an ideal generated by finitely many low-degree polynomials. Applying a transition to a generator lands back in that ideal, possibly modulo an equality guard. For a weighted fold, the object is a finite-dimensional space of residual state vectors. Appending a letter keeps the state in that space, and the output vanishes throughout it. For a definite sum, a finite linear combination of parameter shifts is a difference in the summation index. Summing cancels the interior and leaves explicit endpoints.

These are not three names for the same implementation. Their expensive steps are different: Gröbner reduction, rational linear algebra, and polynomial coefficient solving with support analysis. Their certificates are different, too. What is shared is that the final induction or summation proof need not reproduce the search.

The strongest integration point is not “try a bigger solver on the original goal”. It is “discover a finite closed representation, prove its bridge to the source expression, and discharge the resulting finite identities”.

1.4 Representative goals

The ideal worker should prove that a state initialized on $y = x^2$ stays on that curve under either $(x, y) \mapsto (x + 1, y + 2x + 1)$ or $(x, y) \mapsto (x^2, y^2)$. The second transition does not conserve the polynomial $x^2 - y$: it multiplies it by $x^2 + y$. That distinction defeats a conservation-only ansatz without making the desired invariant difficult.

The weighted worker should prove, for every binary word w ,

$$\#ab(w) + \#ba(w) = \#a(w) \#b(w),$$

where $\#ab$ and $\#ba$ count ordered two-letter subsequences, not adjacent substrings. One side is a five-state

affine fold; the other is a tensor product of two counters. The proof comes from a rank-five closure in a nine-dimensional difference representation.

The telescoping worker should discover and certify, for $S(n) = \sum_{k=0}^n \binom{n}{k}^3$,

$$(n+2)^2 S(n+2) - (7n^2 + 21n + 16)S(n+1) - 8(n+1)^2 S(n) = 0.$$

It should not accept a rational summation certificate merely because its generic formula simplifies: the moving endpoints intersect apparent poles. The explicit regularization and boundary checks in Section 4 are part of the capability, not optional cleanup.

2 Worker I: discover jointly inductive polynomial ideals

2.1 From conservation to preservation of a zero set

Suppose a state is $x = (x_1, \dots, x_n)$ and the transition is a polynomial map $T : \mathbb{Q}^n \rightarrow \mathbb{Q}^n$. A conserved polynomial satisfies the identity $p \circ T = p$. An invariant equation only needs

$$p(x) = 0 \implies p(T(x)) = 0.$$

These are substantially different requirements. For $p = x^2 - y$ and $T(x, y) = (x^2, y^2)$,

$$p(T(x, y)) = x^4 - y^2 = (x^2 + y)(x^2 - y). \quad (1)$$

The equation $p = 0$ is preserved, but the value of p generally changes. A constant-multiplier ansatz would still miss this example; its multiplier really is a polynomial in the state.

A family of equations can be mutually supporting. If $T(x, y) = (y, x)$, then the family $p_1 = x$, $p_2 = y$ is preserved on the zero state, although neither generator is conserved. The appropriate acceptance language is

$$p_i(T_e(x)) = \sum_{j=1}^r A_{eij}(x) p_j(x) + \sum_{\ell=1}^{q_e} B_{ei\ell}(x) g_{e\ell}(x), \quad (2)$$

where edge e is enabled only when all its equality guards $g_{e\ell}(x) = 0$ hold. The matrices A_e and B_e have polynomial entries.

Forge's existing structural section already identifies this more general identity and correctly warns that simultaneously discovering the p_j and A_{eij} gives a bilinear problem [2]. The algorithm below does not solve that simultaneous system by pretending it is linear. It freezes a candidate space, computes ideals generated by that known space, and then solves a genuinely linear preimage problem. Repeating this operation removes unsupported candidates until the space stabilizes.

2.2 Input fragment and semantics

The executed prototype accepts a finite family of polynomial initial maps

$$I_a : \mathbb{Q}^{m_a} \longrightarrow \mathbb{Q}^n$$

and finitely many guarded polynomial transitions (T_e, G_e) , where $G_e = \{g_{e1}, \dots, g_{eq_e}\}$. It seeks a proof that a supplied goal polynomial h vanishes at every state reachable from any $I_a(t)$ by enabled transitions.

Zero-parameter maps represent concrete initial states. Multiple initial maps represent a finite union of parameterized sets.

The initial-map description is a sufficient, explicit source fragment. It does not accept an arbitrary algebraic initial variety and silently replace that variety by sampled points. For a Lean application, the front end must prove that the source initial states are covered by these maps. A map allowed to range over all rationals may be an over-approximation of natural initial parameters; that loses power but not validity for the covered source states.

The prototype treats equality guards only. An inequality guard can be omitted as an over-approximation, so long as the resulting stronger step obligation is actually proved. Exploiting inequalities requires a different certificate, perhaps using Forge's existing ordered-polynomial worker; it is not achieved by turning an inequality into a polynomial equation.

All search is over $\mathbb{Q}$. The resulting identities can be evaluated in a commutative $\mathbb{Q}$ -algebra, such as $\mathbb{R}$. Transport to natural-number code requires proved cast and update equations. Natural subtraction is not rational subtraction, and truncating integer division is not field division.

2.3 The descending-space algorithm

Let $P_d \subset \mathbb{Q}[x_1, \dots, x_n]$ be the vector space of polynomials of total degree at most d . Its dimension is $N = \binom{n+d}{d}$. Start with

$$V_0 = \{p \in P_d : p(I_a(t)) = 0 \text{ identically for every } a\}. \quad (3)$$

A monomial basis of P_d makes this a rational kernel computation: substitute each initial map into each monomial, collect coefficients in its own parameter ring, stack the resulting linear equations, and take the nullspace. Separate initial maps remain separate blocks; their coefficients must not be added to each other.

For a subspace V , let $J_e(V) = \langle V, G_e \rangle$ be the polynomial ideal generated by a basis of V and the equality guards of edge e . Define

$$\Phi(V) = \{p \in V : p \circ T_e \in J_e(V) \text{ for every } e\}. \quad (4)$$

Then compute $V_{k+1} = \Phi(V_k)$ until it stabilizes.

For each edge, compute a Gröbner basis of the *known* ideal $J_e(V_k)$ using a fixed global monomial order. Reduction modulo that basis defines a $\mathbb{Q}$ -linear map

$$L_{e,k} : V_k \rightarrow \mathbb{Q}[x], \quad p \mapsto \text{NF}_{J_e(V_k)}(p \circ T_e).$$

Although the codomain polynomial ring is infinite-dimensional, only finitely many monomials occur in the images of the finite basis. Collecting their coefficients gives a finite matrix. The joint kernel of these matrices is V_{k+1} .

```

V := kernel of all initial-substitution coefficient matrices
repeat:
  residuals := []
  for each transition e:
    G := GroebnerBasis(basis(V) ++ equalityGuards(e))
    residuals.append([normalForm(p(T_e), G) for p in basis(V)])
  K := rationalKernel(coefficientsOfEachResidualBlock(residuals))
  W := linearCombinations(basis(V), columns(K))
  if dimension(W) = dimension(V): break with V := W
  V := W
extract step multipliers against basis(V) ++ guards(e)
extract goal multipliers against basis(V)
return a candidate only if all independently checked identities hold

```

The implementation uses dimension equality because $W \subseteq V$ is known by construction. No equality of pretty-printed basis expressions is required. A production implementation can put each basis in a canonical row-reduced coefficient form to improve caching, but canonicalization is not needed for the termination argument.

Theorem 2.1 (Finite descent). *In the absence of resource cutoffs, the iteration has at most $\dim V_0$ strictly decreasing rounds. At termination its output V_* satisfies $p \circ T_e \in \langle V_*, G_e \rangle$ for every $p \in V_*$ and every edge e .*

Proof. The operator maps subspaces to subspaces because substitution, quotient projection, and coefficient extraction are linear. Also $\Phi(V) \subseteq V$. Every strict inclusion of finite-dimensional vector spaces lowers dimension by at least one. When dimensions agree, the subspaces agree; the defining membership conditions then hold throughout V_* . $\square$

The bound counts *outer rounds*, not arithmetic operations, Gröbner pairs, intermediate monomials, or coefficient bits. An individual round can be expensive. The prototype therefore has distinct basis, polynomial, pair, and round limits. Hitting one raises a resource-cutoff result; it does not produce an empty fixed point or a non-invariance verdict.

2.4 What kind of completeness is obtained?

Definition 2.2. An admissible degree- d inductive generator space is a subspace $W \subseteq P_d$ such that $W \subseteq V_0$ and

$$p \circ T_e \in \langle W, G_e \rangle \quad (p \in W, e \text{ an edge}).$$

Polynomial multipliers have no degree cap in this mathematical definition.

Theorem 2.3 (Greatest admissible space). *With exact completed ideal computations and no resource cutoff, V_* contains every admissible degree- d inductive generator space.*

Proof. Let W be admissible. Initially $W \subseteq V_0$. Suppose $W \subseteq V_k$. For $p \in W$ and every e ,

$$p \circ T_e \in \langle W, G_e \rangle \subseteq \langle V_k, G_e \rangle.$$

Thus $p \in \Phi(V_k) = V_{k+1}$. Induction gives $W \subseteq V_k$ at every round, and therefore $W \subseteq V_*$ when the iteration stops. $\square$

This theorem is stronger than “some template happened to work”, but weaker than completeness for semantic polynomial invariance. Vanishing on a zero set need not coincide with membership in its defining ideal; radical consequences and real-algebraic consequences can require additional reasoning. An invariant may also need generators of degree greater than d , a better state encoding, or a non-polynomial assertion. The initial maps and equality-guard fragment impose further restrictions.

The final goal is accepted when $h \in \langle V_* \rangle$ and a multiplier representation is extracted. The degree of h itself need not be at most d . A high-degree goal can follow from low-degree generators. Conversely, a goal outside this ideal does not become false. The result is simply unknown for this route.

2.5 Multiplier extraction without trusting the Gröbner engine

A normal-form result of zero is useful for search but is not a suitable standalone replay artifact. The output must say how the reduced polynomial is built from the original generators.

The prototype uses an extended Buchberger calculation. For original generators $f_1, \dots, f_s$, each current basis element carries a polynomial vector $u_i \in \mathbb{Q}[x]^s$ satisfying

$$b_i = \sum_j u_{ij} f_j.$$

If an S-polynomial is $ab_i - cb_j$, its accompanying vector is $au_i - cu_j$. Subtracting a multiple qb_ℓ during reduction subtracts qu_ℓ from the vector. Normalizing a nonzero remainder to monic form normalizes its vector by the same scalar. These operations retain exact representations throughout the computation.

After reducing a requested polynomial f to zero, $f = \sum_i q_i b_i$, substitute the recorded vectors to obtain $f = \sum_j (\sum_i q_i u_{ij}) f_j$. Only the latter vector is serialized. The replayer expands the resulting polynomial identity using an independently implemented sparse polynomial representation. It does not rerun Buchberger and does not trust the claimed basis to be Gröbner, reduced, minimal, or even linearly independent.

For larger inputs, a Sage/Singular backend is a sensible replacement for this small educational search. Sage documents Gröbner bases and ideal membership through its Singular-backed multivariate polynomial interfaces [15]. The adapter must also request original-generator representations and validate their orientation by multiplication. A solver’s matrix convention is not part of Forge’s mathematics: the adapter must prove that its decoded multipliers satisfy exactly Equation (2).

2.6 The finite certificate and its soundness

For invariants $p_1, \dots, p_r$, the certificate contains their rational coefficient lists, the multiplier rows A_{eij}, B_{eil} for each transition, and goal multipliers $c_1, \dots, c_r$. The original problem is supplied *separately*. Acceptance verifies

$$p_i(I_a(t)) = 0 \quad \text{for every } i, a, \quad (5)$$

$$p_i(T_e(x)) = \sum_j A_{eij} p_j + \sum_\ell B_{eil} g_{\ell} \quad \text{for every } i, e, \quad (6)$$

$$h(x) = \sum_j c_j(x) p_j(x). \quad (7)$$

Every equality is a formal polynomial identity, not an equation checked on selected points.

Theorem 2.4 (Invariant-certificate soundness). *If the finite checks above hold, then $h(x) = 0$ at every reachable state of the encoded transition system.*

Proof. At an initial state, Equation (5) makes every generator zero. Suppose they are zero at a reachable state and edge e is enabled. Every term on the right of Equation (2) vanishes, using the current invariant equations and the edge’s guards. Hence every generator is zero after the transition. Induction on the number of transitions proves this conjunction at every reachable state. Finally Equation (7) makes the goal vanish. $\square$

The proof uses a conjunction of all generators as its induction predicate. There is no circular admission of one conjectured lemma by another: the *whole conjunction* has an independently checked initial case and step. This is an important difference from installing mutually dependent auxiliary lemmas as if each were already known.

2.7 Examples that discriminate between mechanisms

Scaling versus conservation. Initialize $(x, y) = (t, t^2)$ and use $T = (2x, 4y)$. The worker finds $p = x^2 - y$ and $p \circ T = 4p$. In the degree-two space, a polynomial conserved under the scaling can have only its constant term: the nonconstant monomials scale by factors $2^a 4^b \neq 1$. Initial vanishing then forces that constant to zero. Thus the conservation subspace is zero while the inductive space is nonzero.

Nonlinear multipliers. With the same initial map and $T = (x^2, y^2)$, the discovered multiplier is $x^2 + y$. This example excludes an implementation that merely generalizes $p \circ T = p$ to $p \circ T = \lambda p$ with a constant λ .

Several possible updates. The same invariant survives both the squaring update and $T_1 = (x + 1, y + 2x + 1)$, since $p \circ T_1 = p$. A single invariant family must pass every encoded edge. Checking only the transition that happened to produce a candidate would be an unsound source restriction.

Equality guards. Initialize $(x, y) = (0, t)$, permit the transition $(x, y) \mapsto (x + y, y + 1)$ only under $y = 0$, and seek $x = 0$. Its step is the simple identity $x + y = 1 \cdot x + 1 \cdot y$. The second term is justified by the guard, not by an invariant about all reachable values of y . The mutation suite removes this guard and verifies rejection of the old certificate.

Finite orbits and a genuine degree boundary. Initialize $x = 0$ and use $T(x) = 1 - x$. At degree one, $V_0 = \text{span}_{\mathbb{Q}}\{x\}$ but the preimage test removes it: $1 - x \notin \langle x \rangle$. The search returns unknown for $x^2 - x = 0$. At degree two, the dimensions are 2, 1, 1, and the fixed space is $\text{span}_{\mathbb{Q}}\{x^2 - x\}$. This example explains why an empty low-degree result does not refute the source theorem.

2.8 Implementation refinements worth making next

The first production optimization should be a good exact coefficient backend, not an enlarged template vocabulary. Keep sparse coefficient vectors, cache the normal forms of basis images within a round, and reuse initial-substitution matrices across degree increases. A stable monomial order makes these caches meaningful. Incremental Gröbner algorithms may exploit related ideals between rounds, but dropping generators can invalidate a previously computed basis, so “reuse” must not mean retaining invalid reductions.

For a program with several control-flow locations, use a separate bounded space V_ℓ at each location. For an edge $e : \ell \rightarrow m$, require every target generator $p \in V_m$ to satisfy $p \circ T_e \in \langle V_\ell, G_e \rangle$. The same monotone-descent argument works on the finite product of the location spaces; the total dimension bounds

strict decreases. This extension is a design here, not part of the executed prototype. It is particularly attractive for mutually recursive functions whose invariants naturally differ by control location.

A second refinement is goal-directed generator selection. Run the complete bounded-space pass at a modest degree, then prune a successful certificate to the generators reachable from its goal-multiplier dependencies. Recheck all remaining step identities after pruning. It is safe to shrink a proved certificate; it is not safe to remove a generator during search merely because its name does not occur in the final goal.

3 Worker II: finite residual closure for weighted behaviors

3.1 Why a recursive equality can be linear algebra

Consider two programs that fold over a word. Their internal states may be different, and syntactic rewriting may not find a useful relation between them. Nevertheless, their observable outputs can have finite-dimensional linear representations. Once those representations are proved correct, equality for *every word* becomes a finite closure problem.

A rational weighted representation over a finite alphabet Σ consists of

$$\alpha \in \mathbb{Q}^{1 \times N}, \quad M_a \in \mathbb{Q}^{N \times N} \ (a \in \Sigma), \quad \beta \in \mathbb{Q}^{N \times 1}.$$

Use row vectors throughout. For $w = a_1 \cdots a_k$, set

$$M_w = M_{a_1} \cdots M_{a_k}, \quad f(w) = \alpha M_w \beta,$$

with $M_\epsilon = I$. This is the usual linear-algebraic view of weighted automata over a field [10, 11]. The relevant extension to Forge is a small certificate-producing worker for this fragment, not a new automata theory.

Two representations can be compared by taking their difference:

$$\alpha = (\alpha_L \ \alpha_R), \quad M_a = \text{diag}(M_a^L, M_a^R), \quad \beta = \begin{pmatrix} \beta_L \\ -\beta_R \end{pmatrix}. \quad (8)$$

It suffices to decide whether $\alpha M_w \beta = 0$ for all w . The dimensions need not agree, and no syntactic correspondence between their state components is assumed.

3.2 Reachable-space search

Let $\mathcal{V} = \text{span}_{\mathbb{Q}}\{\alpha M_w : w \in \Sigma^*\}$. This is a subspace of $\mathbb{Q}^{1 \times N}$, so it has dimension at most N . A breadth-first worklist constructs a basis from reachable rows. Each row carries a word that reaches it.

```

queue := [(alpha, emptyWord)]
basis := []
while queue is not empty:
    (v, w) := popFront(queue)
    if v * beta != 0:
        return DistinguishingWord(w)
    if v is in span(basis):
        continue
    append v to basis
    for each letter a:
        pushBack(queue, (v * M[a], w ++ [a]))
extract alpha coordinates and transition coordinates in basis
return ClosureCertificate(basis, coordinates)

```

The span test is exact rational elimination. There are at most N independent rows, and each independent row enqueues at most $|\Sigma|$ successors. Thus at most $1 + |\Sigma|N$ queue rows are examined when no counterexample or budget cutoff ends the run early. Dependent rows do not need expansion because the successors of a linear combination are the same linear combinations of the successors.

Theorem 3.1 (Complete finite search over a field). *For a finite-dimensional rational weighted representation and no resource cutoff, the algorithm returns either a word w with $f(w) \neq 0$ or a finite closure certificate proving $f(w) = 0$ for every word.*

Proof. Each appended row increases the dimension of the current span, so only finitely many are appended and the queue terminates. Every appended row is reachable. On termination, the span contains α and is closed under every right multiplication M_a : successors of every basis row were processed and found in the final span. Induction on word length puts every αM_w in that span. Every basis row has zero product with β , so every reachable row does as well. If the algorithm stops earlier on a nonzero product, its stored word is an explicit counterexample. $\square$

Proposition 3.2 (Bounded distinguishing words). *If the represented series is nonzero, some distinguishing word has length at most $N - 1$.*

Proof. Let V_j be the span of rows reached by words of length at most j . If $\alpha = 0$, the series is zero. Otherwise $\dim V_0 = 1$. Whenever $V_j = V_{j+1}$, that space is closed under every transition and hence contains all reachable rows. Before stabilization, its dimension increases strictly. It therefore stabilizes by $j = N - 1$. If the output is zero on all rows through that length, it is zero on their span and thus on all words. $\square$

The proof does not say that brute-force enumeration through length $N - 1$ is a good algorithm: that enumeration is exponential in N for a nontrivial alphabet. The reachable-space search avoids that expansion by recognizing linear dependence.

3.3 The closure certificate

Let the rows of $B \in \mathbb{Q}^{r \times N}$ span a suitable closed space. Supply a row $c_0 \in \mathbb{Q}^{1 \times r}$ and matrices $C_a \in \mathbb{Q}^{r \times r}$ such that

$$\alpha = c_0 B, \quad BM_a = C_a B \quad (a \in \Sigma), \quad B\beta = 0. \quad (9)$$

The replay checks are only matrix equalities over exact rationals.

Theorem 3.3 (Weighted-closure soundness). *Equation (9) implies $\alpha M_w \beta = 0$ for all words w .*

Proof. Writing $C_w = C_{a_1} \cdots C_{a_k}$, induction on the word gives

$$\alpha M_w = c_0 C_w B.$$

The empty case is the initial-coordinate identity. The extension step is

$$c_0 C_w B M_a = c_0 C_w C_a B.$$

Multiply the resulting equation by β and use $B\beta = 0$. $\square$

Replay does *not* need a rank computation. The rows of B can be redundant, and even a larger non-reachable closed space can certify the result if the three identities hold. Independence is needed only for the efficiency and termination analysis of the search. Keeping it out of the certificate checker makes the proof principle smaller.

The zero-initial-vector case admits $r = 0$: the initial-coordinate equality then forces $\alpha = 0$, and the closure and output conditions are empty. The empty alphabet case has just the empty word and requires no transition identities. Both cases are covered by the executed tests. Matrix dimensions and the exact alphabet inventory are checked explicitly, so omitting a difficult letter is not an allowed certificate simplification.

A counterexample certificate is smaller still: it contains a word, and replay computes its nonzero output from the original representation. This is a genuine negative answer in the encoded fragment. It transfers to a source program only after the source-to-representation bridge is proved, just as a positive answer does.

3.4 Building representations from code

Linear folds. If a state update is $v \mapsto vM_a$ and the output is $v\beta$, the representation is direct. The front end must prove the equations connecting these coordinates to the source fold. It should not infer those equations from the shape of pretty-printed code.

Affine folds. For $v \mapsto vA_a + b_a$, adjoin a constant coordinate and represent the state as $z = (1, v)$. Then

$$z \mapsto z \begin{pmatrix} 1 & b_a \\ 0 & A_a \end{pmatrix}.$$

The invariant that the first coordinate is one follows from the matrix form. Affine outputs are handled by the same coordinate. This covers many counters, linear accumulators, and statistics of words.

Scalar recurrences. A constant-coefficient linear recurrence has a companion-matrix state and a one-letter alphabet. Equivalence of two recurrence implementations is therefore an instance of the same worker. Polynomial coefficients depending on the index are not constant matrices; either a proved finite lift exists or another worker, such as telescoping, is required.

Addition and products of observations. Direct sums represent addition. For pointwise products over $\mathbb{Q}$,

$$\begin{aligned} \alpha_{fg} &= \alpha_f \otimes \alpha_g, \\ M_a^{fg} &= M_a^f \otimes M_a^g, \\ \beta_{fg} &= \beta_f \otimes \beta_g. \end{aligned}$$

The Kronecker-product identity gives $f(w)g(w) = \alpha_{fg}M_w^{fg}\beta_{fg}$. This permits nonlinear *output* identities even though the expanded state dynamics remain linear. Repeated tensor lifting can multiply dimensions rapidly, so it requires an explicit state-dimension budget and subsequent reachable-space reduction.

Finite observations of an infinite alphabet. A source type need not literally be finite if a proved classifier maps each input element to one of finitely many transition classes and the update depends only on that classifier. The bridge theorem must establish this factoring. Without it, checking a finite sample of letters does not establish a word identity over an infinite alphabet.

3.5 Worked example: a product of counts versus subsequence counts

For a binary word, let A, B count occurrences of a, b , and let AB, BA count ordered subsequences of length two. The row state is

$$z = (1, A, B, AB, BA).$$

Appending a updates A to $A + 1$ and BA to $BA + B$; appending b updates B to $B + 1$ and AB to $AB + A$. All other coordinates remain unchanged. Starting from $(1, 0, 0, 0, 0)$, the output $AB + BA$ is therefore a five-dimensional weighted representation.

A two-dimensional affine counter represents A , and another represents B . Their tensor product has four coordinates and represents AB as the *product of counts*. To avoid a notational ambiguity in an implementation, call that output $A \cdot B$, reserving `countABSubseq` for the subsequence counter.

The difference construction has ambient dimension nine. The executed search finds five reachable basis rows, reached by

$$\epsilon, \quad a, \quad b, \quad ab, \quad ba,$$

and examines eleven worklist rows. Its certificate verifies

$$\forall w \in \{a, b\}^*, \quad \text{countABSubseq}(w) + \text{countBASubseq}(w) = \text{countA}(w) \text{countB}(w)$$

for the encoded folds. A separate simple combinatorial proof pairs every a -position with every b -position and partitions the pairs by their order. That proof explains the identity, but it is *not* supplied to the search. The prototype derives the finite linear relation instead.

The example exercises several capabilities at once: different source states, noncommuting letter matrices, a tensor-composed right side, and an invariant space smaller than the combined coordinate space. Replacing both sides by a single known algebraic expression would not test those capabilities.

3.6 Counterexamples, minimization, and use during synthesis

A minimal state representation is not required to decide zero equivalence. It can nonetheless be useful after success. First restrict to the reachable space using B and C_a . Then compute which residual directions affect future outputs, quotienting only after proving the corresponding observation map. Classical weighted-automaton minimization provides the algebraic background for this second phase [10]. The prototype implements reachable-space closure, not full minimal realization.

During term synthesis, an exact distinguishing word can reject a candidate whose proved weighted summary disagrees with the requested weighted specification. A proved match can discharge *that observable contract*. Neither result justifies replacing equality of arbitrary source functions by equality of their lengths or counts. For example, two list-valued functions can have equal length on all inputs while returning different elements in a different order. The result type of the adapter should name the observable it has established.

This provides a useful bridge to Leant and Djex: their current semantic foundation already associates behavioral analyses with source-owned typed candidate graphs [7]. A weighted domain can analyze a new class of behaviors without altering the candidate's type certificate or claiming that the existing length abstraction suddenly became complete.

3.7 Cost model and boundaries

With incremental rational elimination, each independence test can be reduced to $O(N^2)$ field operations, and each dense successor multiplication also costs $O(N^2)$. Across $O(|\Sigma|N)$ generated rows, a straightforward

optimized implementation has an $O(|\Sigma|N^3)$ field-operation bound. Bit complexity also depends on coefficient growth. Fraction-free or modular linear algebra with exact reconstruction is a production option; reconstruction must still pass the finite equalities in Equation (9).

The included SymPy implementation recomputes linear solves for span membership and coordinate extraction. It is intentionally simpler than an incremental cubic implementation, so its measured times must not be presented as evidence for that optimized bound. Its role is to validate the algorithm, certificate, and source examples.

The field assumption matters for the decision algorithm. Rational weighted zero equivalence is not Boolean-language equivalence with the word “rational” erased, and it is not arbitrary-semiring equivalence. A nondeterministic weighted representation may count accepting paths rather than indicate their existence. A Lean bridge must preserve which semantics is being modeled.

4 Worker III: pole-safe creative telescoping

4.1 Beyond polynomial antidifferences

Forge’s polynomial recurrence worker solves identities such as $q(n+1) - q(n) = p(n)$ for polynomial p [2]. That mechanism is well suited to sums of powers. It does not, by itself, cover a summand like $\binom{n}{k}^3$, where the parameter n changes both the summand and the range of summation.

Creative telescoping instead seeks a linear recurrence in n whose summand is a difference in k . The method is classical; reduction-based algorithms also provide compact certificates for more general D-finite summands [12]. For Forge, the useful deliverable is not an opaque recurrence guessed by a CAS. It is a recurrence together with finite identities and explicit support obligations sufficient to prove it in Lean.

The prototype specializes to

$$F_m(n, k) = \binom{n}{k}^m, \quad S_m(n) = \sum_{k=0}^n F_m(n, k), \quad n \in \mathbb{N},$$

with fixed positive integer m . The mathematical convention is that $\binom{n}{k} = 0$ for integer $k < 0$ or $k > n$. A Lean front end can implement this zero extension explicitly, or avoid negative indices through a separate $k = 0$ branch. It must not interpret $k - 1$ as truncated natural subtraction in a formula whose derivation assumed an integer index.

4.2 A fully explicit bounded ansatz

Choose an order $r \geq 1$, a coefficient-degree bound a , and a total-degree bound b for a bivariate numerator. Search for

$$p_0(n), \dots, p_r(n) \in \mathbb{Q}[n], \quad \deg p_j \leq a, \quad U(n, k) \in \mathbb{Q}[n, k], \quad \deg U \leq b,$$

with $p_r \neq 0$, satisfying

$$\begin{aligned} \sum_{j=0}^r p_j(n) \left(\prod_{t=1}^j (n+t) \right)^m \left(\prod_{t=j+1}^r (n+t-k) \right)^m \\ = (n+r-k)^m U(n, k+1) - k^m U(n, k). \end{aligned} \tag{10}$$

Empty products are one. The certificate equation is an identity in the polynomial ring $\mathbb{Q}[n, k]$.

Write each unknown polynomial in the specified monomial basis and equate coefficients. This is a homogeneous rational linear system with

$$(r+1)(a+1) + \binom{b+2}{2}$$

unknowns. A nullspace basis describes all solutions in that ansatz. If any solution has nonzero p_r , at least one nullspace basis vector does. Common polynomial factors, rational denominators, and integer content can then be removed consistently from all p_j and U to obtain a compact representative.

```

for a scheduled order r and degree bounds a, b:
  introduce coefficients for p[0..r](n) and U(n,k)
  form the difference of the two sides of the polynomial identity
  M := its exact coefficient matrix
  for a nullspace basis vector with p[r] != 0:
    normalize common factors and rational content
    regularize the summation certificate
    check lower boundary and every upper support boundary
    check the leading coefficient and initial values
    return the finite certificate if all checks succeed
return UNKNOWN for the exhausted scheduled ansatz

```

The implementation is a bounded coefficient search, not a full implementation of Gosper's or Zeilberger's algorithms. A failed ansatz is not evidence that no recurrence exists. The exact linear system is complete for its chosen coefficient space; the scheduled degree bounds and the final leading-coefficient recognizer impose additional restrictions.

4.3 Where the apparent poles come from

Introduce

$$D(n, k) = \prod_{t=1}^r (n+t-k), \quad A(n) = \prod_{t=1}^{r-1} (n+t).$$

A conventional rational certificate suggested by the ansatz is

$$R(n, k) = \frac{k^m U(n, k)}{D(n, k)^m}.$$

Formally, away from poles, one would use $G(n, k) = R(n, k)F_m(n, k)$ and seek

$$\sum_{j=0}^r p_j(n) F_m(n+j, k) = G(n, k+1) - G(n, k).$$

But $D(n, k)$ vanishes at $k = n+1, \dots, n+r$, precisely where the shifted summands can still be nonzero. Substituting a pole and claiming that a zero-valued F_m cancels it is invalid. In Lean, where field division is total, blind evaluation can be even more misleading: the totalized expression is not the removable extension used in the mathematical derivation.

The remedy is to change the certificate representation before specializing it. Define the *regularized* quantity

$$H(n, k) = U(n, k) \binom{n+r-1}{k-1}^m. \quad (11)$$

It is defined for every integer k by the zero-extension convention. The identity we actually prove is

$$A(n)^m \sum_{j=0}^r p_j(n) F_m(n+j, k) = H(n, k+1) - H(n, k). \quad (12)$$

No rational pole appears in this statement.

4.4 Deriving the interior identity without illegal cancellation

For $1 \leq k \leq n$, all factors of $D(n, k)$ are positive integers. Factorial identities give

$$\begin{aligned} \frac{\binom{n+j}{k}}{\binom{n}{k}} &= \frac{\prod_{t=1}^j (n+t)}{\prod_{t=1}^j (n+t-k)}, \\ \frac{\binom{n+r-1}{k-1}}{\binom{n}{k}} &= \frac{kA(n)}{D(n, k)}, \\ \frac{\binom{n+r-1}{k}}{\binom{n}{k}} &= \frac{A(n)(n+r-k)}{D(n, k)}. \end{aligned}$$

These identities are used only where their denominators are nonzero. In particular, the last two are applied directly to H ; they do not evaluate $R(n, k+1)$ at its possible pole when $k = n$.

Divide the proposed identity by the nonzero $\binom{n}{k}^m$, substitute these ratios, and multiply by $D(n, k)^m$. The result is $A(n)^m$ times Equation (10). Since $A(n) \neq 0$ for $n \geq 0$, the polynomial identity establishes the interior case. In a Lean proof, the same argument should preferably be organized as cross-multiplied factorial lemmas, with the positivity hypotheses explicit, rather than as a sequence of loosely controlled rational simplifications.

4.5 Boundary equations are part of the certificate

The ratio argument does not cover $k = 0$ or $k = n+t$ for $1 \leq t \leq r$. These are handled symbolically, not by evaluating finitely many numerical n .

At $k = 0$, every $F_m(n+j, 0) = 1$, $H(n, 0) = 0$, and $H(n, 1) = U(n, 1)$. The required identity is therefore

$$A(n)^m \sum_{j=0}^r p_j(n) = U(n, 1). \quad (13)$$

At $k = n+t$, only shifts $j \geq t$ contribute. Symmetry of binomial coefficients reduces them to fixed-lower-index polynomials:

$$\binom{n+j}{n+t} = \binom{n+j}{j-t}.$$

For a fixed nonnegative integer q , the expression

$$\binom{n+c}{q} = \frac{1}{q!} \prod_{u=0}^{q-1} (n+c-u)$$

is a rational polynomial in n on the required natural domain. The two boundary terms from H similarly involve $\binom{n+r-1}{r-t-1}$ and $\binom{n+r-1}{r-t}$, with a negative lower index interpreted as zero. Thus each of the r upper-boundary conditions becomes a univariate rational polynomial identity.

The independent checker explicitly expands and verifies all these identities. Their cost is small compared with the coefficient search, and they prevent the most obvious mismatch between generic hypergeometric algebra and the intended finite sum.

Theorem 4.1 (Pole-safe summation certificate). *Assume Equation (10), the lower-boundary identity, and all r upper-boundary identities hold. Then for every $n \in \mathbb{N}$,*

$$\sum_{j=0}^r p_j(n) S_m(n+j) = 0. \quad (14)$$

Proof. The interior and boundary arguments prove Equation (12) for every k in the common range $0 \leq k \leq n+r$. Sum over that range. Each shifted summand $F_m(n+j, k)$ vanishes beyond $n+j$, so its sum is exactly $S_m(n+j)$. The right side telescopes to

$$H(n, n+r+1) - H(n, 0) = 0,$$

since the upper binomial coefficient has lower index $n+r$ and upper index $n+r-1$, while the lower endpoint has lower index -1 . The left side is $A(n)^m$ times Equation (14). Every factor of $A(n)$ is positive, and the empty product when $r=1$ is one. Cancel this nonzero scalar. $\square$

No asymptotic limit or infinite summation theorem is needed. The difficulty is uniform handling of a moving finite support, not convergence.

4.6 A recurrence is not yet an equality with a proposed sequence

To characterize a sequence uniquely from an order- r recurrence, require $p_r(n) \neq 0$ for every $n \geq 0$ and supply the first r values. If sequences S and T share those initial values and satisfy the same recurrence, their difference D satisfies the homogeneous recurrence. Strong induction gives $D(n), \dots, D(n+r-1) = 0$, after which $p_r(n)D(n+r) = 0$ forces $D(n+r) = 0$.

The prototype accepts a simple sufficient nonvanishing certificate: either p_r or $-p_r$ has a strictly positive constant coefficient and all coefficients nonnegative. More sophisticated examples should route this obligation to Forge's existing univariate sign machinery rather than weaken the condition. The initial values are computed by finite exact binomial sums.

There are consequently two different useful outputs. A *recurrence proof* establishes Equation (14). A *sequence-characterization proof* adds nonvanishing and initial data. The prototype's acceptance profile requires the latter side conditions even when only the recurrence is being recorded. A production interface should expose these as different result constructors.

The mutation tests multiply every p_j and U by $n-2$. This preserves the interior and boundary polynomial identities and produces a true recurrence, but its leading coefficient vanishes at $n=2$. The checker reaches and rejects the nonvanishing step. Such a recurrence is not false; it simply does not justify the same unrestricted uniqueness argument.

4.7 Three synthesized certificates

First powers. For $m=1$, choose $r=1, a=0, b=0$. The three-equation system in three unknowns has a one-dimensional nullspace. Normalization gives

$$p_0 = -2, \quad p_1 = 1, \quad U = -1.$$

Thus $S_1(n+1) = 2S_1(n)$ and $S_1(0) = 1$. Ordinary induction gives $S_1(n) = 2^n$. The recorded artifact certifies the recurrence and its initial data; this last closed-form identification is the additional mathematical argument just given.

Squares. For $m = 2$, use $r = 1, a = 1, b = 1$. The nine-equation system in seven unknowns yields

$$p_0 = -2(2n+1), \quad p_1 = n+1, \quad U = 2k - 3n - 3.$$

The regularized certificate is particularly small:

$$H(n, k) = (2k - 3n - 3) \binom{n}{k-1}^2.$$

It proves

$$(n+1)S_2(n+1) = 2(2n+1)S_2(n), \quad S_2(0) = 1. \quad (15)$$

For example, the upper boundary at $k = n+1$ has left side $n+1$ and right side $0 - (-(n+1)) = n+1$. The lower boundary gives $-3n-1$ on both sides.

The sequence $\binom{2n}{n}$ has the same initial value and ratio $2(2n+1)/(n+1)$, as follows from factorials. Equation (15) and uniqueness therefore prove the familiar identity $S_2(n) = \binom{2n}{n}$. The factorial ratio is a source-level proof obligation, not a string returned by the telescoper and automatically trusted.

Cubes. For $m = 3$, the executed search uses $r = 2, a = 2, b = 5$: thirty unknowns, forty-five coefficient equations, and a one-dimensional nullspace. It finds

$$\begin{aligned} p_0(n) &= -8(n+1)^2, \\ p_1(n) &= -7n^2 - 21n - 16, \\ p_2(n) &= (n+2)^2, \end{aligned}$$

and $U(n, k) = (n+1)^2 P(n, k)$, where

$$\begin{aligned} P(n, k) &= 4k^3 - 18k^2n - 30k^2 + 27kn^2 + 93kn + 78k \\ &\quad - 14n^3 - 74n^2 - 128n - 72. \end{aligned}$$

Here $A(n) = n+1$ and

$$H(n, k) = (n+1)^2 P(n, k) \binom{n+1}{k-1}^3.$$

The resulting Franel-number recurrence is

$$(n+2)^2 S_3(n+2) = (7n^2 + 21n + 16) S_3(n+1) + 8(n+1)^2 S_3(n), \quad (16)$$

with $S_3(0) = 1$ and $S_3(1) = 2$. This is a certified recurrence, not a claim that the algorithm found a single hypergeometric closed form for S_3 .

All three certificates are stored in `results/certificates.json` with the original summand-family parameters. The checker verifies the bivariate identity, $r+1$ boundary rows, leading-coefficient positivity, and the initial values. Additional tests compare the recurrences against direct finite sums for $n = 0, \dots, 30$; those numerical regressions are not substituted for the symbolic proof obligations.

4.8 A route to a broader summation worker

The next implementation layer should separate summand recognition from operator search. Recognize a source summand only after proving its shift relations and support description. Then permit a more capable search backend to propose an operator and certificate, while retaining the same source-level obligations for summation.

For hypergeometric antidifferences, SymPy documents `gospers_normal`, `gospers_term`, and `gospers_sum` [14]. For D-finite operator manipulation, Sage’s `ore_algebra` provides creative telescoping, closure operations, transformations, guessing, and desingularization [16]. The reduction-based work of Brochet and Salvy is a source of algorithms for compact definite-summation certificates [12]. These are proposed backends, not dependencies exercised by the included tests.

Operator serialization must fix a normal form. For example, write $L = \sum_j p_j(n) E_n^j$ with coefficients on the left, where $E_n f(n) = f(n+1)$. The rule $E_n n = (n+1)E_n$ means that ordinary commutative polynomial multiplication is not a correct implementation of operator composition. A recurrence checker that merely evaluates the listed coefficient polynomials at the current n avoids this ambiguity; a broader operator engine must preserve it explicitly.

Finally, support handling should become a first-class component. A general summand may have different moving lower and upper bounds, internal poles, exceptional small parameters, or a nonzero boundary contribution. In those cases the output recurrence can be inhomogeneous. The worker must return the boundary term and prove it, not discard it because the generic certificate was computed successfully.

5 Concrete integration into Forge, Leant, and Djex

5.1 Reuse the existing controller; add three semantic front ends

Forge already has proposed obligation, candidate, and conditional-candidate contracts. In particular, its runtime specimen has a `ConditionalCandidate` carrying premises and a constructor term, and an `EngineReply` that can return progress or a reasoned failure [5]. The workers in this report fit that contract. They do not require a replacement for Forge’s scheduler or an embedded second Lean kernel.

The missing layer is a *proved extraction* from a source expression into one of the three fragments. A certificate about an independently invented transition system is not a proof of the user’s recursive function. The following proposed module split isolates this work:

Forge/Behavior/PolynomialState.lean	-- source transition descriptions
Forge/Behavior/Weighted.lean	-- affine/linear fold descriptions
Forge/Behavior/BinomialSum.lean	-- summand and finite-support bridges
Forge/Certificate/InductiveIdeal.lean	-- finite identities and soundness
Forge/Certificate/Weighted.lean	-- closure matrices and word replay
Forge/Certificate/Telescoping.lean	-- regularized summation, uniqueness
Forge/Worker/InductiveIdeal.lean	-- problem extraction and CAS adapter
Forge/Worker/Weighted.lean	-- exact reachable-space search
Forge/Worker/Telescoping.lean	-- coefficient search and proof assembly
Forge/Export/BehaviorProof.lean	-- stable theorem/lemma output

These are proposed files, not files asserted to exist in the reviewed repository. The supplied Python files implement the corresponding search and finite replay functions; the Lean source front ends remain to be implemented.

5.2 Polynomial extraction and replay

For the first version, accept explicit polynomial-state descriptions rather than arbitrary recursive programs. A description contains an exact coordinate telescope, initial polynomial maps, update polynomials, equality-guard polynomials, a goal polynomial, and Lean proof obligations tying these to the source.

For an ordinary recursive function, derive its transition view from checked equations. A natural-number loop can package its loop index into the state, for example $(i, s) \mapsto (i + 1, F(i, s))$. A list fold can use one update per finite letter class. The equation compiler must use simultaneous substitution: under $(x, y) \mapsto (y, x)$, both right sides refer to the old state. A sequential substitution routine would silently implement a different transition.

Mathlib’s `MvPolynomial` evaluation interface provides `eval`, `evalâĈĈ`, `evalâĈĈHom`, and `aeval`; evaluation and substitution lemmas support the mathematical interpretation of coefficient identities [18]. A replay theorem can evaluate each serialized polynomial in the source coordinates, use the checked identities to prove the invariant conjunction’s step, and apply ordinary reachability induction. The core-only `reachableInvariant` specimen in this package shows the last induction principle independently of any polynomial implementation.

There are two practical replay implementations. For a narrow first release, translate each coefficient list to an ordinary Lean polynomial expression, prove the finite identities with `ring` and rational normalization, and apply the generic soundness theorem. For a faster later release, implement a small reflected sparse-polynomial evaluator and prove its correctness once. Both routes end in a Lean proof term. The latter is an optimization of repeated identity checking, not a prerequisite for testing the mathematical fragment.

Avoid making a proof of Gröbner-basis correctness an early dependency. The final certificate never requires that fact. It only requires original-generator identities. The uncompiled `ReplayTargets.lean` file includes the concrete nonlinear-curve identity and its invariant step, illustrating the small output that the search can leave behind.

5.3 Weighted extraction and replay

Start with an explicit weighted representation over `Rat`. Its dimensions should be type-indexed by `FinN` and `Finr`, not represented by unchecked array lengths inside the mathematical theorem. The external decoder first establishes the lengths, then constructs the corresponding Lean values. The theorem consumes B, c_0, C_a and proofs of the three identities in Equation (9).

Mathlib documents `Matrix.vecMul`, `Matrix.mulVec`, and associativity lemmas including `Matrix.vecMul_vecMul` [19]. These give a straightforward row-vector proof. The supplied `simulateWord` specimen abstracts the argument: if an embedding commutes with each letter step, then it commutes with running any word. For the matrix certificate, that embedding is $c \mapsto cB$, and the commuting square is $BM_a = C_aB$.

The first source front end should support a finite enumerated alphabet and explicit tuples of rational affine accumulators. Syntactic recognition can propose the matrices, but Lean must prove the source step equals the proposed affine map for every letter and state. After that bridge is established, the worker can compare independently structured folds, not just normalized copies of one expression.

The negative route returns a concrete word and a proved inequality between its source observations. For source programs with executable data, this can be shown as a diagnostic even when the goal itself is not a proposition asserting universal equality. It does not authorize a global non-inhabitation conclusion about an unrelated synthesis type.

5.4 Summation extraction and replay

The initial sum front end should recognize exactly

$$\sum_{k \in \text{range}(n+1)} (\text{Nat.choose}(n, k) : \mathbb{Q})^m$$

for a literal supported m . It can later add source-equivalent spellings by proved rewrites. Recognition must preserve the cast location: a natural power cast into $\mathbb{Q}$ can be related to the rational power by a theorem, but neither expression should be guessed from surface syntax.

Useful Mathlib ingredients include `Nat.choose_eq_zero_of_lt`, `Nat.choose_pos`, `Nat.choose_symm`, and `Nat.choose_mul_factorial_mul_factorial` [20]. The last is especially useful for cross-multiplied ratio proofs, avoiding premature field division. Finite-sum lemmas in the `Finset` big-operator API provide the reindexing and induction infrastructure [21]. The package does not assume an existing theorem with exactly the pole-safe certificate signature; that theorem is one of the explicit implementation tasks.

Prove the regularized summation principle once for arbitrary r, m, p, U satisfying the finite obligations. Then a particular certificate supplies only rational polynomial identities, fixed boundary-index calculations, positivity data, and initial values. To prove equality with another expression, attach a second recurrence proof and invoke the shared uniqueness principle. Do not recompute the entire sum for a large numerical range during Lean replay.

The concrete Lean specimen includes the first-power, square, and cube *interior polynomial identities*. It does not contain a full Lean proof of the binomial support bridges or Equation (16). Keeping this boundary explicit prevents the short algebraic specimen from being mistaken for a completed formalization of the sum theorem.

5.5 A practical worker transaction

A complete worker invocation should behave as follows. The frontend first constructs a typed fragment problem and the bridge obligations. It calls a search backend with the problem’s immutable coefficient data. The result is decoded into finite certificate data. The appropriate replay theorem proves the fragment result. Finally, a proof constructor combines that result with *all* bridge proofs to produce the original goal.

```
runBehaviorWorker(sourceGoal):
  fragment, bridges := extractSupportedFragment(sourceGoal)
  candidate := search(fragment)
  if no candidate: return NoResult(reason)
  finiteProof := replayFiniteCertificate(fragment, candidate)
  if bridges are already proved:
    return Candidate(transport(finiteProof, bridges))
  return ConditionalCandidate(
    premises = bridges,
    constructor = transportConstructor(finiteProof))
```

This is pseudocode for the proposed adapter, not a claim about an existing API function named `runBehaviorWorker`. Forge’s current controller vocabulary already has the needed conditional shape [5]. The new content is the fragment extraction and the replay theorems, not the existence of a conditional work item.

The result should usually be exported as a local or named lemma that ordinary `grind` can consume. The current Lean reference describes `grind` as a combination of equality and theory reasoning producing proof

terms [8]. The new lemma changes the facts available to that engine: a global invariant, a fold equivalence, or a sum recurrence is information that a larger local arithmetic budget would not necessarily discover.

5.6 Concrete library choices

Layer	Choice	Exact role and boundary
Prototype search	Python 3.13.5; SymPy 1.14.0	Polynomial expansion, rational coefficient matrices, exact nullspaces; tracked Buchberger search supplied in this package. Executed.
Independent replay	Python standard library <code>fractions.Fraction</code>	Canonical rational and sparse polynomial data, matrix arithmetic, exact finite checks. Runs with <code>python-S</code> . Executed.
Larger ideal search	Sage/Singular	Use <code>I.groebner_basis()</code> and <code>f.lift(I.gens())</code> ; export original-generator multipliers and verify their identity. Proposed, not executed here [15].
Larger linear algebra	FLINT-backed exact arithmetic	Replace repeated rational solves with fraction-free or modular computations, then reconstruct and recheck the same closure identities. Proposed [17].
Broader summation	SymPy Gosper; Sage <code>ore_algebra</code>	Antidifferences or operator/certificate proposals; preserve source support and boundary proofs. Proposed [14, 16].
Lean replay	Lean and pinned Mathlib	Prove polynomial interpretation, finite-space simulation, binomial support, telescoping, and uniqueness. Not run in this package.

A useful Sage adapter can be very small: create the exact rational polynomial ring with a fixed variable order, construct I from the original generators, request `f.lift(I.gens())`, and serialize the returned polynomial list. The documentation explicitly illustrates reconstructing f by multiplying that list by the generator list [15]. Failure to lift is a search failure; a malformed or incorrect returned identity is replay rejection. There is no need to transmit a complete Gröbner trace to Lean.

The archive does not bundle these external systems or their licenses. Its own code is released under MIT-0. Backend packaging and license compatibility should be handled separately from the mathematical certificate format; the format itself does not depend on the chosen search library.

5.7 Leant and Djex: add behavior domains, not another candidate authority

Djex’s documented semantic layer consumes a source-owned typed candidate graph rather than manufacturing a new term identity. Its current length domain tracks a supported list-spine behavior through checked problem descriptions and replay [7]. Two additional domains fit this arrangement:

Polynomial-state contracts assign a proved polynomial transition summary to a source fold or recursive accumulator. The ideal worker can prove postconditions about that state. *Weighted-observation contracts*

assign a proved weighted representation to a scalar observable. The weighted worker can prove equality or return a finite distinguishing input for that observable.

Each summary must carry its full source type, the exact chosen instances and providers, and the theorem connecting the candidate’s semantics to the representation. These are not extra versions of the type certificate. They are additional theorems about the same candidate. When a provider is polymorphic, instantiate the summary at the same source-level type and universe selections as the retained candidate graph; substituting a conveniently similar monomorphic provider changes the problem.

The benefit is more than post-hoc scoring. A verified summary can discharge a behavioral obligation for all inputs in its fragment. A counterexample can reject a candidate quickly. But neither domain should prune unrelated higher-order alternatives merely because their current partial terms lack an extractable summary. Unsupported analysis remains absence of information.

5.8 A targeted composition experiment for the remaining indexing gap

Leant’s current diagnosis is unusually concrete. The isolated integer-indexing fold step is found at candidate 5, and the same step in a larger outer context at candidate 219, while the original full indexing query still fails all 256 checked candidates in its recorded window [6]. These are not equivalent benchmark tasks, and the diagnosis explicitly leaves full indexing unaccepted.

A focused follow-on experiment is to introduce a *typed composition cut*. When a candidate skeleton contains a function-valued fold, expose its step and base as named synthesis obligations using their exact source types. Search the step obligation first under a dependency-closed subset of the local telescope, and retain a successful step as a candidate component. Reinsert it by applying the original fold provider and independently checking the complete term.

The dependency-closed telescope is computable: start from variables occurring in the hole’s type, explicit behavioral contract, and selected provider arguments; repeatedly add free variables occurring in the types and local values of those variables, including instance dictionaries. Search this reduced context as an additional lane, not as a completeness-preserving replacement for the full-context lane. A solution found in the reduced context is valid in the full context after checked weakening; failure in the reduced context proves nothing about the larger query.

For the documented step type

```
forall a. a -> a -> (Int -> a) -> Int -> a
```

a reusable component should still distinguish negative, zero, and positive indices and retain the supplied default behavior. Cache the generalized checked component under its complete source type and provider inventory. Do not reuse an ordinal such as “candidate 5” as an identity, and do not assume the same component cost in the outer search: usage history and pending holes can change that cost.

This experiment is proposed, not implemented or accepted here. Its success criterion is the *original* indexing query at its recorded signature, behavior observations, candidate window, and limits, with the exact displayed term replayed. It is included because the repository now identifies a specific composition bottleneck to investigate, rather than because another generic suggestion to improve ranking would add much value.

6 Executed experiments and what they establish

6.1 Recorded environment and reproducibility

The package records one ordered local run with seed 20260915, Python 3.13.5, SymPy 1.14.0, and pytest 9.0.2 on Linux x86-64. Search and independent replay are timed separately using a monotonic performance counter. The raw records are in `results/experiments.json`; the full original problems and certificates are in `results/certificates.json`. Reproduction writes to a new directory by default rather than overwriting those records.

The search implementations do not import the checker. They use SymPy expressions and matrices. The checker uses its own sparse polynomial dictionaries and standard-library rational arithmetic, without SymPy. A separate invocation `python-Sreplay.py` accepts all 78 stored artifacts with site packages disabled. This is implementation diversity at the finite arithmetic boundary, not a proof that the checker contains no bug.

Worker	Attempts	Positive certificates	Counterexamples	Unknown
Inductive ideals	29	27	0	2
Weighted representations	48	23	25	0
Binomial telescoping	5	3	0	2

These are counts of different artifact types, not a theorem-proving success rate. In particular, the twenty-five weighted counterexamples are successful *inequivalence decisions*, while an ideal unknown is not a counterexample. The problem families are deliberately constructed to exercise mechanisms and rejection boundaries; they are not an unbiased sample of Lean developments.

6.2 Invariant corpus and ablation

The ideal corpus contains nine named controls and twenty generated curve families. The generated cases apply random small rational-coordinate changes to a parameterized quadratic curve and construct polynomial transitions that preserve it with a nonconstant multiplier. This gives planted, mathematically known invariants while varying the expanded coefficients and update syntax. The search is still given the initial map and transition, not the planted invariant as a basis element.

The restricted ablation solves only global conservation equations $p(T_e) = p$ together with initial vanishing. It ignores guards as an aid to the conservation search, so it is deliberately narrower than a full implication-based invariant method. The important observations are qualitative:

Case	Conserved dim.	Inductive dim.	Descent dimensions
Curve scaling	0	1	1, 1
Nonlinear squaring	0	1	1, 1
Two transition branches	0	1	1, 1
Mutual swap	1	2	2, 2
Guarded update	0	1	1, 1
Finite orbit, degree two	1	1	2, 1, 1
Finite orbit, degree one	0	0	1, 0
False invariant control	0	0	2, 1, 0

For the mutual-swap case, the one-dimensional conservation space contains $x + y$, which does not imply the desired $x = 0$. The two-dimensional inductive space contains both x and y , and does imply it. This is why comparing only whether the two spaces are nonempty would miss an important distinction.

The false-invariant control starts at zero and increments the state while asking for $x = 0$. Its empty fixed point is consistent with the false target, but the worker still reports unknown rather than pretending its restricted search proved a counterexample. The degree-one finite-orbit miss is a true target with the same unknown status. Together they enforce the intended result semantics.

6.3 Weighted corpus

The weighted corpus contains the subsequence-count identity, twenty change-of-basis equivalence pairs of component dimensions two through five, twenty perturbed pairs that agree on the empty word but disagree later, five nilpotent-chain examples, and the zero-initial and empty-alphabet edge cases.

For the equivalence pairs, a rational invertible matrix P transforms

$$(\alpha, M_a, \beta) \mapsto (\alpha P, P^{-1} M_a P, P^{-1} \beta).$$

These pairs are equivalent by construction, but their matrices are not syntactically equal. The perturbations change the final vector in a direction orthogonal to the initial row, ensuring that a check of only the empty word cannot distinguish them. Every returned nonempty word is independently reevaluated.

The five one-letter nilpotent chains of dimensions three through seven have first distinguishing word lengths two through six, respectively. They attain the $N - 1$ bound and test that the implementation does not cut search short after several zero outputs. The separate differential tests enumerate all words up to length $N - 1$ in sixty randomly generated representations of dimensions one through four, comparing the complete finite oracle with the closure search.

6.4 Telescoping corpus

Family	r	a	b	Unknowns	Equations	Result
$\binom{n}{k}$	1	0	0	3	3	Certificate
$\binom{n}{k}^2$	1	1	1	7	9	Certificate
$\binom{n}{k}^3$	2	2	5	30	45	Certificate
$\binom{n}{k}^2$, narrow	1	0	0	3	6	Unknown
$\binom{n}{k}^3$, order one	1	2	2	12	21	Unknown

The successful systems each have nullity one. The two narrow systems have nullity zero. This demonstrates the importance of recurrence order and numerator degree, but it does not establish a generally optimal scheduling policy or a minimal-order theorem for every summand family.

The certificate checker accepts powers one through eight and orders one through five as data bounds, but only the three named positive families above were synthesized and recorded. Decoder support for a parameter range must not be reported as experimental success on every member of that range.

6.5 Mutation and differential tests

The test suite reports **318 passed**. Its decomposition is useful:

Test class	Count
Replay of stored original-problem/certificate pairs	78
Rejection controls, including semantic and input mutations	148
Tracked-Gröbner differential cases	24
Weighted exhaustive differential cases	60
Additional direct-sum recurrence regressions	3
Bounded misses, degree distinction, budgets, import separation	5

The 148 rejections include changing an ideal goal while keeping its old certificate, altering a transition multiplier, altering initial-span or closure coordinates, replacing an actual distinguishing word by a nondistinguishing one, corrupting telescoping numerators or initial values, and introducing a singular leading coefficient while preserving all algebraic identities. They also include malformed rationals, booleans used as integers, floating-point JSON tokens, duplicate object keys, duplicate or unordered polynomial terms, missing guards, missing letters, and an unsupported summand family.

The twenty-four Gröbner cases compare ideal-membership behavior with SymPy’s Gröbner implementation and independently expand extracted original-generator lifts. The search basis need not be reduced, so the test does not incorrectly require two different normal-form implementations to print identical remainders. It checks agreement on zero membership and equivalence of remainders modulo the ideal.

These controls test meaningful failure modes, but the decoder is still a research implementation. Its byte, term, dimension, exponent, and input-bit limits are not a complete defense against every expensive intermediate arithmetic computation. A production worker should enforce wall-clock, allocation, and total-operation budgets in addition to validating data.

6.6 Recorded costs, without a tactic-speed claim

The sum of per-case search times in the one recorded run was approximately 0.571 seconds for ideals, 0.421 seconds for weighted representations, and 0.319 seconds for telescoping. The corresponding sums of independent replay times for returned artifacts were 0.026, 0.040, and 0.012 seconds. These times exclude process startup and are not repeated-run estimates.

Compact JSON certificate payloads ranged from 134 to 202 bytes for ideal certificates; from 46 to 887 bytes for weighted artifacts, where many are short counterexample words; and from 139 to 504 bytes for the three telescopers. Those sizes exclude the original problem descriptions, pretty-printing, and future Lean proof terms. The full JSON corpus is larger because it retains those original problems and readable indentation.

The numbers establish that the small examples and their finite checks are practical in this environment. They do not predict Gröbner performance on large programs, optimized matrix performance, Lean elaboration time, kernel checking cost, or success rates against `grind`, `Aesop`, `LeanHammer`, or any other tactic. None of those head-to-head experiments was performed.

6.7 The remaining evidence gap

The executed work establishes implementation behavior for the included Python fragments, finite certificate replay, and concrete counterexamples to some restricted precursors. The written proofs establish why the specified certificates imply their encoded mathematical claims.

The unfilled gap is source formalization: compile the generic Lean soundness principles, implement proved source bridges, generate actual theorem terms, and replay them under the pinned Lean/Mathlib environment.

The two supplied Lean files are specimens for that work. Their presence is not evidence that those steps have already succeeded.

7 An implementation order with falsifiable acceptance gates

7.1 First gate: one complete weighted proof in Lean

The weighted worker is the most economical first production slice. Its search is elementary exact linear algebra, its certificate has three equations, and its source fragment can initially be explicit rational matrices. Implement the matrix closure theorem and a decoder that constructs dimension-indexed Lean values. Then replay the stored subsequence-product certificate against an explicit Lean definition of the two folds and a proved affine/tensor bridge.

The acceptance gate is not “the matrices simplify”. It is a theorem over *all source words*, produced from the certificate, compiled in the pinned environment, with the exact final theorem and axiom inventory retained. Change one transition coefficient, omit a letter, and alter one closure entry; the same replay path must reject each stale artifact. A distinguishing-word test must produce a Lean-checked inequality on an actual source input.

Only after that gate should the source front end recognize a larger family of folds automatically. Starting with explicit representation annotations isolates semantic extraction from linear-algebra debugging and makes failures easier to locate.

7.2 Second gate: a non-conserved invariant from source code

Port the invariant soundness theorem and the coefficient-expression replay route. Use the nonlinear squaring curve as the minimum positive example, with a natural-number iteration count and a rational state. The output theorem must show the invariant for every iteration, not merely prove the one-step identity.

Require a second test with two possible transitions and a third with an equality guard. Those prevent the implementation from accidentally specializing to single-map conservation. Retain the degree-one finite-orbit miss as an unknown control and the degree-two success as a separate acceptance. Do not make the frontend’s degree bound depend on seeing the target theorem’s known proof.

For the CAS adapter, start with the included Python worker or the documented Sage lifting interface. The final Lean proof should depend on the decoded multipliers, not on which external search produced them. Changing the search backend should require new performance tests, not a new trusted theorem.

7.3 Third gate: the square sum with all endpoints

Formalize the zero-extended binomial semantics or an equivalent piecewise natural-index formulation. Prove the regularized summation theorem. Then use the recorded square-sum certificate to prove the recurrence, prove the candidate sequence $\binom{2n}{n}$ satisfies the same recurrence, and apply uniqueness.

The negative gates should include the old rational expression evaluated at a pole, a dropped endpoint contribution, a wrong range upper bound, and the $(n - 2)$ -multiplied recurrence offered as an unrestricted uniqueness certificate. The latter is especially valuable because its polynomial algebra is correct; it tests semantic completeness of the acceptance obligations rather than whether ring can detect a typo.

Next, replay the cube-sum recurrence and its two initial values. This acceptance must remain labeled as a recurrence theorem unless a separate target sequence has been proved to share the recurrence. A

general-summation milestone should add at least one nonzero-boundary example before expanding the recognized summand grammar substantially.

7.4 Controller integration and proof export

Initially run each worker as a bounded child of the existing obligation controller. Use the conditional-candidate path when source bridges remain. Once a worker proves a useful consequence, assert that consequence as an ordinary theorem and let the baseline tactic continue. An incremental grind extension is not necessary to validate the new mathematics.

Export should preserve three pieces: the source bridge theorem, the finite certificate data, and the theorem applying the soundness principle to those data. Ordinary recompilation should not need the external solver. A compact local lemma is usually preferable to replaying search inside the final tactic script.

For observability, attach worker-specific diagnostics rather than only a single “failed” counter. The ideal worker should report the degree, descent dimensions, Gröbner budget status, and whether goal lifting failed. The weighted worker should report ambient dimension, independent row count, examined row count, and any distinguishing word. The telescoping worker should report order, degree caps, coefficient-matrix size, nullity, and the first unproved support or nonvanishing obligation. These are directly actionable reasons to change a budget or repair a source bridge.

7.5 Evaluation after the formalized slice exists

Build a held-out Lean corpus grouped by the reason the worker should help: nonconserved invariant families, independently implemented affine folds, tensor-composed observations, constant-coefficient recurrences, and definite hypergeometric sums with moving support. Include true unsupported goals and false variants in each family. Record source theorem statements before tuning the worker and exclude target theorems or equivalent lemmas from retrieval.

Compare the baseline tactic under the same imports, premises, and resource limits with the baseline plus each worker individually and all workers combined. Also compare against a simple portfolio of existing tactics, since an advantage over a deliberately weak baseline would not establish the value of the new algorithms. Record search time, elaboration time, kernel checking time, proof size, solver startup, and replay-only cost separately.

The important ablation is semantic. Remove polynomial multipliers from the ideal certificate language. Disable tensor composition from the weighted frontend. Disable support regularization from the sum worker while retaining its strict rejection of undefined obligations. These variants test the incremental capabilities described in this report. Merely lowering a generic search budget would not identify which mathematical mechanism caused success.

7.6 Further directions that fit the same boundaries

The most natural next invariant extension is a product of location-specific spaces, followed by equality-aware treatment of piecewise polynomial updates. For stronger semantic consequences, radical or real-radical certificates could be added as a separate proof language; they should not be smuggled into ordinary ideal membership.

For weighted behavior, add symbolic finite classifiers, several simultaneous scalar observations, and proved reachable/observable minimization. For a vector output, the certificate condition becomes $BB_{\text{out}} = 0$ with

several output columns, so the same closure search supports multiple contracts at once. Polynomial outputs may be tensor-lifted selectively, but a dimension budget is essential.

For sums, support inhomogeneous recurrences, exceptional initial parameter ranges, and certificates expressed by a small operator basis rather than a single hypergeometric ratio. A broader CAS backend should increase search power while leaving the Lean summation obligations explicit.

These directions should follow, rather than postpone, the three complete Lean acceptance gates. The present prototypes already provide enough exact data to make those gates concrete.

7.7 Assessment

The proposed capability increase is specific. Forge would acquire a method for finding invariant *equations* that are not conserved quantities; a complete finite-dimensional equivalence method for rational weighted behaviors; and a method for deriving uniform recurrences for non-polynomial finite sums without ignoring their support. Each returns small mathematical objects that ordinary induction or finite summation can consume.

The most valuable next artifact is a compiled source-level theorem using one of these certificates. The package makes that artifact substantially more specified: the finite data, independent replay, mathematical soundness proof, source fragment, relevant library interfaces, and rejection controls are all present. What remains is the Lean implementation and its measurement, not an unspecified request to make automation more powerful.

A Artifact map and reproduction

A.1 Files and their roles

Path	Role
prototype/ideal.py	Descending spaces, tracked Buchberger algorithm, original-generator multiplier extraction, conservation comparator.
prototype/weighted.py	Reachable-space search, difference construction, affine/tensor subsequence example.
prototype/telescoping.py	Bounded linear coefficient synthesis for binomial-power telescopers.
prototype/check.py	Independent standard-library exact replayer and bounded canonical-data decoder.
prototype/encode.py	SymPy-side serialization, not imported by the checker.
run_experiments.py	Seeded corpus construction and separate search/replay measurements.
replay.py	Standalone replay of stored original problems and certificates.
tests/test_certificates.py	318 recorded pytest cases, including semantic mutations and differential controls.
results/certificates.json	78 original-problem/certificate pairs.

Path	Role
results/experiments.json	Full per-attempt outcomes, dimensions, coefficients, and local timings.
results/stdlib-replay.json	Receipt from replay with site packages disabled.
results/pytest.xml, results/pytest.log	Test report and captured console output.
results/status.json	Repository pins and explicit implemented/not-run status.
lean/ProofPrinciples.lean	Uncompiled core proof-principle specimens.
lean/ReplayTargets.lean	Uncompiled concrete Mathlib polynomial replay targets.
article/forge-finite-certificates.tex	Main report source; section and reference files are in the same directory tree.

A.2 Commands

Use Python 3.10 or newer. The recorded environment was Python 3.13.5. From the package root:

```
python -m pip install -r requirements.txt
python -m pytest -q
python -S replay.py
python run_experiments.py --out reproduced-results
python -S replay.py reproduced-results/certificates.json
```

The fourth command regenerates the cases; it does not overwrite the supplied measurements. Timings will vary. The deterministic inputs and outcomes should match under the recorded dependency version; serialized basis choices should not be used as a cross-version semantic identity.

Build the report from `article/` with three passes:

```
pdflatex -interaction=nonstopmode -halt-on-error forge-finite-certificates.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-finite-certificates.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-finite-certificates.tex
```

The main source uses ordinary TeX Live packages, including New TX text/math, Microtype, Listings, and Tcolorbox. Font files are not included in the archive.

To try the Lean specimens, place them inside an independently configured, pinned Lean/Mathlib project and invoke `lakeenvlean` on each file. This is a reproduction instruction, not a record that the commands were executed here. Their `NOT_RUN` headers are intentional.

B Certificate data contracts

B.1 Rationals and sparse polynomials

A rational is the pair $[a, b]$ of exact JSON integers with $b > 0$ and $\gcd(|a|, b) = 1$. A zero coefficient is represented by $[0, 1]$ when it occurs in a vector or matrix; zero terms are omitted from polynomials. A polynomial in n variables is a strictly lexicographically ordered list of [exponent vector, rational coefficient]

pairs. For example, $x^2 - y$ in variable order (x, y) is

```
[[[0,1],[-1,1]], [[2,0],[1,1]]]
```

The zero polynomial is the empty list. Exponent dimension and variable order come from the original problem, not from a claimed theorem name in the certificate.

B.2 Inductive-ideal data

The problem contains `dimension`, initial maps with explicit parameter counts, transitions with update and guard polynomials, and the goal polynomial. The certificate contains `invariants`, `steps`, and `goal_multipliers`. For transition e and invariant i , `steps[e][i]` is a list of multipliers in the order

$$(p_1, \dots, p_r, g_{e1}, \dots, g_{eq_e}).$$

Thus the checker can validate multiplier count and meaning without any Gröbner-basis data. Initial maps are interpreted separately even if they use the same number of parameter variables.

B.3 Weighted data

The source problem supplies `dimension`, `alphabet`, `alpha`, `beta`, and one matrix for every alphabet label. A closure certificate supplies `rank` (the number of rows supplied, not a replayed rank claim), `basis`, `initial_coordinates`, and `closure`. A negative certificate supplies `word`. The two kinds have different acceptance conditions: finite matrix identities versus a directly reevaluated nonzero output.

B.4 Telescoping data

The source problem fixes `family=binomial_power_sum` and `power`. The certificate contains `order`, univariate `coefficients`, the bivariate `numerator`, and exact `initial_values`. Its intended domain is all natural n , with zero-extended binomial support. The polynomial identity and regularization formulas are fixed by the versioned certificate kind, not provided as executable text by the search backend.

C Audit scope and source provenance

The repository audit focused on Forge’s README, structural and nonlinear sections, conclusion, provenance appendix, neighboring-project integration notes, and runtime design specimen. Leant and Djex were inspected through their current READMEs, source-owned semantic description, and the latest indexing composition diagnosis. These sources identify what the consolidated design already covers and where the present increment begins.

The report does not reproduce the nine historical proposals, claim an exhaustive absence search across every preserved file, or incorporate their old experimental counts into its new results. Every count in Section 6 comes from this package’s own executed corpus. Mathematical background is attributed in the references; the detailed certificate theorems and derivations are stated and proved in the report.

References

- [1] Vladimir Reshetnikov, *Forge*, README and repository, snapshot 674521027d968d59f7b83220ed52304a85cb55e2, inspected 15 September 2026. <https://github.com/VladimirReshetnikov/Forge/tree/674521027d968d59f7b83220ed52304a85cb55e2>.
- [2] Forge, *Structural search*, consolidated article, source article/sections/05-structural.tex, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/article/sections/05-structural.tex>.
- [3] Forge, *Provenance: what came from where*, source article/sections/B-provenance.tex, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/article/sections/B-provenance.tex>.
- [4] Forge, *Ideas from Leant and Djex*, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/docs/IDEAS-FROM-LEANT-DJEX.md>.
- [5] Forge, *Runtime design contract*, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/lean/Forge/Design/Runtime.lean>.
- [6] Vladimir Reshetnikov, Leant, *Indexing composition after native kind adoption*, 14 September 2026, snapshot 6bf05ad78c467989e68290f2d08bbbed40802d485. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/reports/2026-09-14-indexing-composition-diagnostics.md>.
- [7] Vladimir Reshetnikov, Djex, *Semantic foundations*, snapshot e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5. <https://github.com/VladimirReshetnikov/Djex/blob/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5/docs/semantic-foundations.md>.
- [8] Lean project, *The grind tactic*, Lean language reference, consulted 15 September 2026. The online “latest” page is not a substitute for a pinned runtime in an experiment. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [9] Sriram Sankaranarayanan, Henny B. Sipma, and Zohar Manna, *Non-Linear Loop Invariant Generation using Gröbner Bases*, POPL 2004. Author’s publication page: <https://theory.stanford.edu/~srirams/papers/popl2004.html>.
- [10] Stefan Kiefer, *Notes on Equivalence and Minimization of Weighted Automata*, arXiv:2009.01217, 2020. <https://arxiv.org/abs/2009.01217>.
- [11] Wen-Guey Tzeng, *A Polynomial-Time Algorithm for the Equivalence of Probabilistic Automata*, SIAM Journal on Computing 21(2), 216–227, 1992. <https://doi.org/10.1137/0221017>.
- [12] Hadrien Brochet and Bruno Salvy, *Reduction-Based Creative Telescoping for Definite Summation of D-finite Functions*, arXiv:2307.07216v2, 2023. <https://arxiv.org/abs/2307.07216v2>.
- [13] SymPy, *Polynomial manipulation reference*, version 1.14.0 documentation, consulted 15 September 2026. <https://docs.sympy.org/latest/modules/polys/reference.html>.
- [14] SymPy, *Concrete mathematics and Gosper functions*, version 1.14.0 documentation, consulted 15 September 2026. <https://docs.sympy.org/latest/modules/concrete.html>.

- [15] SageMath, *Ideals in multivariate polynomial rings*, documentation of Gröbner bases and original-generator lifting; consulted 15 September 2026. https://doc.sagemath.org/html/en/reference/polynomial_rings/sage/rings/polynomial/multi_polynomial_ideal.html.
- [16] SageMath, *ore_algebra: Ore algebra*, package documentation; consulted 15 September 2026. https://doc.sagemath.org/html/en/reference/spkg/ore_algebra.html.
- [17] FLINT project, *Fast Library for Number Theory*, project and documentation. <https://flintlib.org/>.
- [18] Mathlib, *Mathlib.Algebra.MvPolynomial.Eval*, generated API documentation, consulted 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Algebra/MvPolynomial/Eval.html.
- [19] Mathlib, *Mathlib.Data.Matrix.Mul*, generated API documentation, consulted 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Data/Matrix/Mul.html.
- [20] Mathlib, *Mathlib.Data.Nat.Choose.Basic*, generated API documentation, consulted 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Data/Nat/Choose/Basic.html.
- [21] Mathlib, *Mathlib.Algebra.BigOperators.Group.Finset.Basic*, generated API documentation, consulted 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Algebra/BigOperators/Group/Finset/Basic.html.